configuration system's built-in capability to bind settings to C# objects. The options pattern is intended to be used to group related configuration values together in individual classes. Rather than relying on `IConfiguration`, you inject `IOptions<TOptions>` into your service class constructor and access its `Value` property, which should return an instance of `TOptions`. Before that, you need to register the `SmtpSettings` configuration section with the service container, using the `Configure<TService>` method.

Listing 14.15 Binding settings using the `Configure` method

```
builder.Services.Configure<SmtpSettings>(
     builder.Configuration.GetSection(nameof(SmtpSettings)));
```

Now the configuration can be injected into constructors using the `IOptions <TOptions>` interface. You will need to add a using directive for `Microsoft.Extensions .Options` to the `ProductionEmailService` class; the only other change will be replacing the injected `IConfiguration` with an `IOptions<SmtpSettings>` and then assigning its `Value` property to the `SmtpSettings` field.

Listing 14.16 Using `IOptions<TOptions>` instead of `IConfiguration`

```
public class ProductionEmailService : IEmailSender
{
    private readonly ILogger<ProductionEmailService> _logger;
    private readonly SmtpSettings _settings;

    public ProductionEmailService(ILogger<ProductionEmailService>
    ➥ logger, IOptions<SmtpSettings> options)                    ❶
    {
        _logger = logger;
        _settings = options.Value;                                ❷
    }
    ....
```